

Structures de données 2

Tableaux, graphes, ensembles

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Tableaux

Représenter des données matricielles

2

Graphes

Différentes représentations informatiques de graphes en utilisant des tableaux

3

Ensembles

Tableaux

Les tableaux sont utiles pour résoudre toutes sortes de problèmes.

- Représenter les positions d'un jeu (échecs, go, etc.)
- Faire des calculs sur des matrices
  - En maths
  - Dans les jeux vidéo (rotations, translations, projections)
  - Au cœur de l'intelligence artificielle
- Représenter des graphes dans un ordinateur
- Etc.

9	13	24
34	45	56

Un tableau de 3 colonnes et 2 lignes

Tableaux

9	13	24
34	45	56

Question · 2 min

Comment représenter ce tableau en Python ?

Vous connaissez déjà tous les éléments nécessaires.

Listes : type des éléments de la liste

Indice	0	1	2	3
Équipe	`"Liverpool"`	`"Marseille"`	`"Olympiacos"`	`"Ajax"`

N'importe quel *type* de données peut être contenu dans une liste.

```
>>> l1 = ["Liverpool", "Marseille", "Olympiacos", "Ajax"]
>>> l1[0]
'Liverpool'

>>> l2 = [4, 6, 7, 11]
>>> l2[0]
4

>>> l3 = [True, False, True, True]
>>> l3[0]
True
```

Tableaux : listes de listes

9	13	24
34	45	56

Ce tableau peut être représenté par une liste de listes.

```
>>> tableau = [[9, 13, 24], [34, 45, 56]]
```

Équivalent, mais plus facile à visualiser :

```
>>> tableau = [[9, 13, 24],
...            [34, 45, 56]]
```

Combien d'éléments contient la liste `tableau` ?

Les éléments de la première liste sont les rangées du tableau.

```
>>> tableau[0]
[9, 13, 24]
```

Exercice · 3 min

Comment accéder au nombre 56 ?

Tableaux : listes de listes

9	13	24
34	45	56

Ce tableau peut être représenté par une liste de listes.

```
>>> tableau = [[9, 13, 24], [34, 45, 56]]
```

Équivalent, mais plus facile à visualiser :

```
>>> tableau = [[9, 13, 24],
...            [34, 45, 56]]
```

Combien d'éléments contient la liste `tableau` ?

Les éléments de la première liste sont les rangées du tableau.

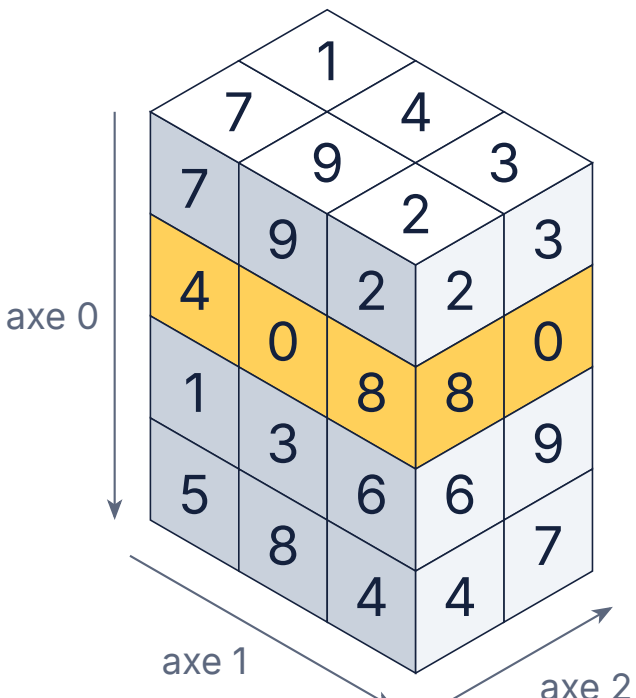
```
>>> tableau[0]
[9, 13, 24]
```

Exercice

Comment accéder au nombre 56 ?

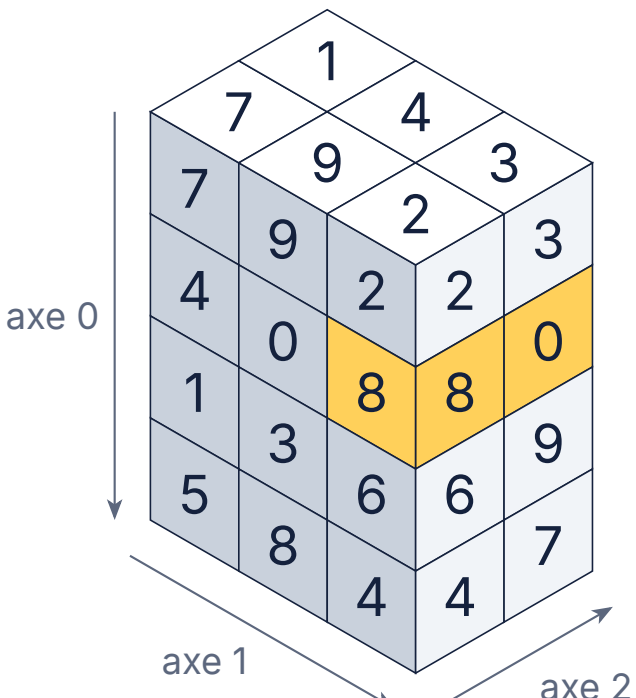
```
>>> tableau[1][2]
56
```

Tableaux : listes de listes de listes



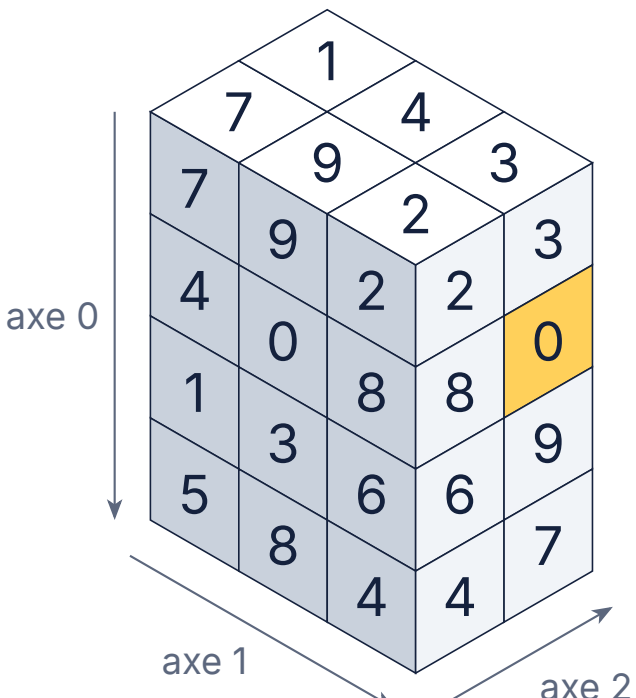

```
>>> tableau[1][2][1]
0
```

Tableaux : listes de listes de listes




```
>>> tableau[1][2][1]
0
```

Tableaux : listes de listes de listes




```
>>> tableau[1][2][1]
0
```

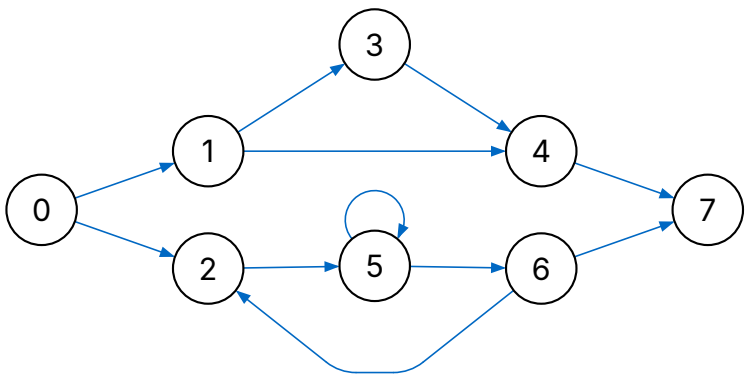
Sommaire

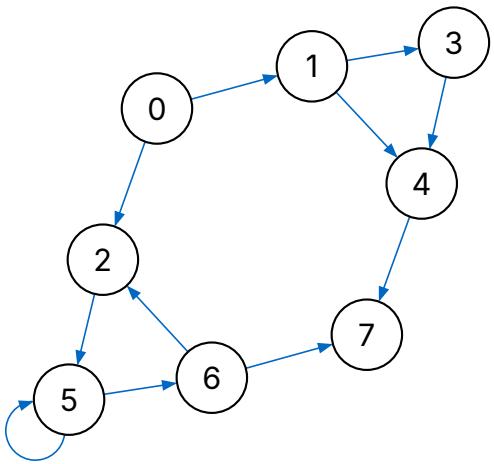
1 **Tableaux**
Représenter des données matricielles

2 **Graphes**
Différentes représentations informatiques de graphes en utilisant des tableaux

3 **Ensembles**

Tableaux : représentation de graphes





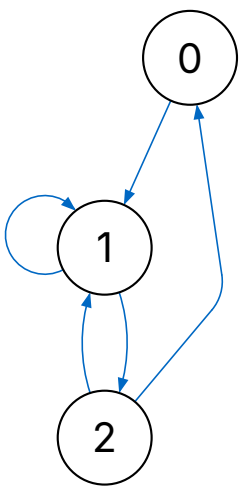
[Graphviz en ligne]

C'est la *structure* d'un graphe qui le définit, pas sa représentation graphique.

Il y a de nombreuses manières de représenter la structure d'un graphe dans un programme. Nous en verrons deux :

- Liste d'enfants
- Matrice d'adjacence

Représentation de graphes : liste d'enfants



Graphviz en ligne

On crée une liste avec autant d'éléments qu'il y a de nœuds.

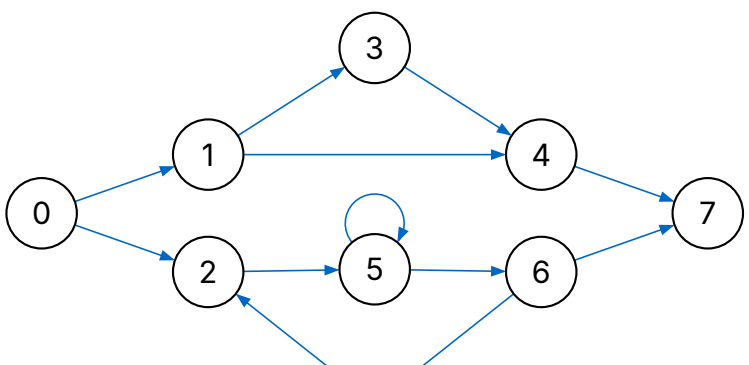
Chaque élément de la liste est à son tour une liste contenant les enfants du nœud.

```
# La représentation du graphe de gauche sous forme
# de tableau Python
graphe = [
    [1],     # Nœud 0 -> arc vers le nœud 1
    [1, 2],  # Nœud 1 -> arcs vers les nœuds 1 et 2
    [0, 1],  # Nœud 2 -> arcs vers les nœuds 0 et 1
]
```

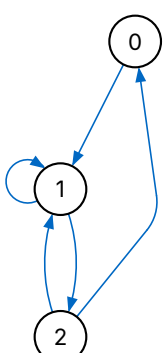
Exercice : graphe comme liste d'enfants

Exercice · 7 min

Écrire le code Python qui fabrique une variable `graphe`, contenant la description de ce graphe sous forme de liste d'enfants.



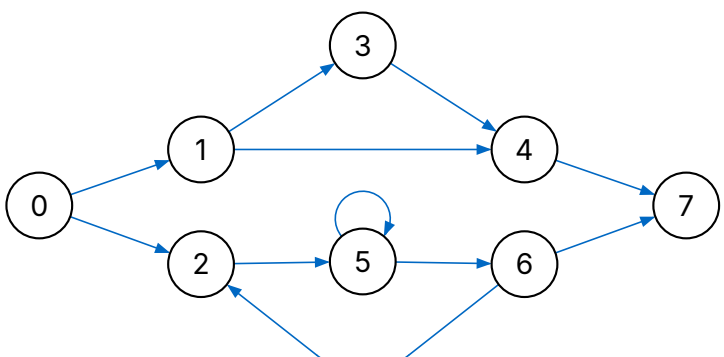
Rappel :




```python
graphe = [
    [1],     # Nœud 0 -> arc vers le nœud 1
    [1, 2],  # Nœud 1 -> arcs vers les nœuds 1 et 2
    [0, 1],  # Nœud 2 -> arcs vers les nœuds 0 et 1
]
```

Exercice : graphe comme liste d'enfants

Écrire le code Python qui fabrique une variable `graphe`, contenant la description de ce graphe sous forme de liste d'enfants.



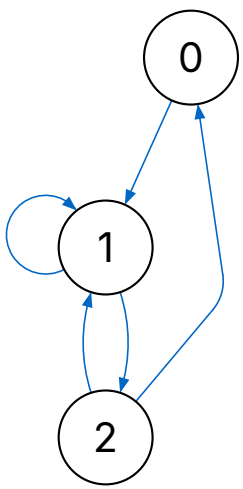

```python
# Définir le graphe sous forme de liste d'enfants
graphe = [
    [1, 2],  # Nœud 0 -> arcs vers les nœuds 1 et 2
    [3, 4],  # Nœud 1 -> arcs vers les nœuds 3 et 4
    [5],     # Nœud 2 -> arc vers le nœud 5
    [4],     # Nœud 3 -> arc vers le nœud 4
    [7],     # Nœud 4 -> arc vers le nœud 7
    [5, 6],  # Nœud 5 -> boucle et arc vers le nœud 6
    [2, 7],  # Nœud 6 -> arcs vers les nœuds 2 et 7
    [],      # Nœud 7 -> pas d'enfants
]

# Afficher le graphe
for noeud, enfants in enumerate(graphe):
    print("Nœud", noeud, "->", enfants)
```

Python en ligne

Représentation de graphes : matrice d'adjacence

Le graphe a 3 nœuds, donc une matrice 3 × 3.
Un 1 dans la matrice en position (r, c) indique qu'il y a un arc de r à c.



	0	1	2
0	0	**1**	0
1	0	**1**	**1**
2	**1**	**1**	0

Code Python correspondant :

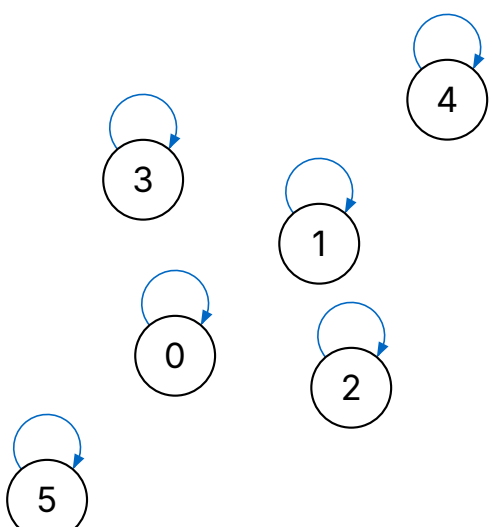
```python
matrice_adjacence = [
    [0, 1, 0],  # Nœud 0 -> arc vers le nœud 1
    [0, 1, 1],  # Nœud 1 -> arcs vers les nœuds 1 et 2
    [1, 1, 0],  # Nœud 2 -> arcs vers les nœuds 0 et 1
]
```

Matrice d’adjacence : boucles

Exercice · 4 min

À quoi ressemble la matrice d’adjacence de ce graphe ?

- Dessiner la matrice (pas besoin de code Python).
- Décrire en une phrase comment on construit la matrice, quel que soit le nombre de nœuds (par exemple 1 million).

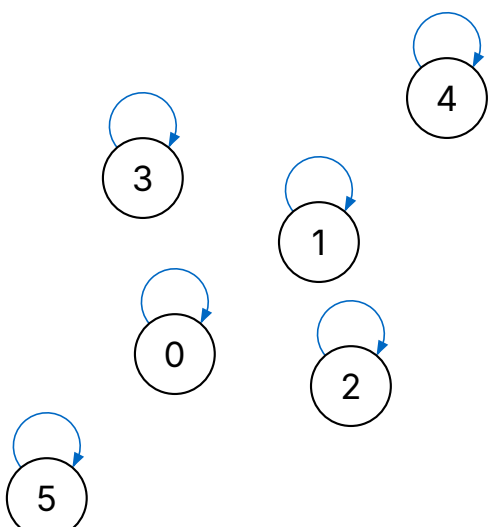


Graphviz en ligne

Matrice d'adjacence : boucles

À quoi ressemble la matrice d'adjacence de ce graphe ?

- *Dessiner la matrice (pas besoin de code Python).*
- *Décrire en une phrase comment on construit la matrice, quel que soit le nombre de nœuds (par exemple 1 million).*



Matrice d'adjacence de ce graphe :

	0	1	2	3	4	5
0	**1**	0	0	0	0	0
1	0	**1**	0	0	0	0
2	0	0	**1**	0	0	0
3	0	0	0	**1**	0	0
4	0	0	0	0	**1**	0
5	0	0	0	0	0	**1**

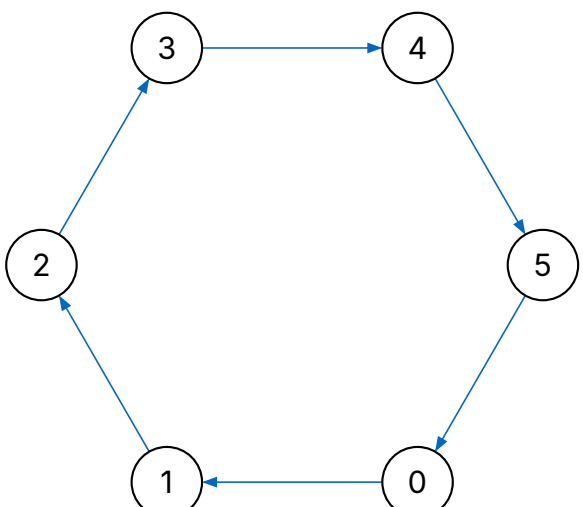
Recette : des 1 sur la diagonale, des 0 partout ailleurs.

Matrice d’adjacence : cycles

Exercice · 3 min

À quoi ressemble la matrice d’adjacence de ce graphe ?

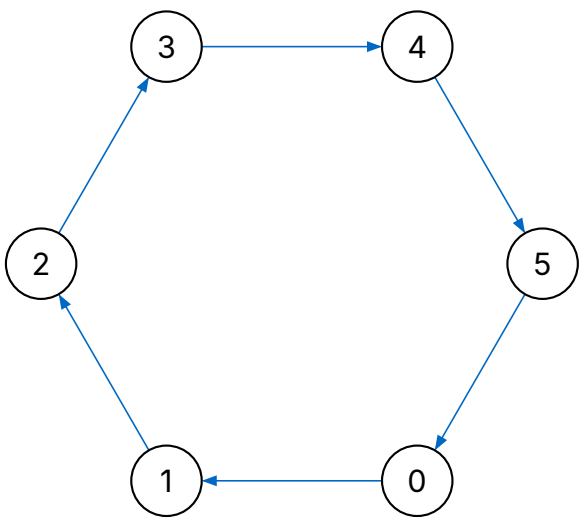
Dessiner la matrice (pas besoin de code Python).



Graphviz en ligne

Matrice d'adjacence : cycles

À quoi ressemble la matrice d'adjacence de ce graphe ?



Matrice d'adjacence de ce graphe :

	0	1	2	3	4	5
0	0	**1**	0	0	0	0
1	0	0	**1**	0	0	0
2	0	0	0	**1**	0	0
3	0	0	0	0	**1**	0
4	0	0	0	0	0	**1**
5	**1**	0	0	0	0	0

Exercice : dessiner le graphe de cette matrice d'adjacence

Exercice · 6 min

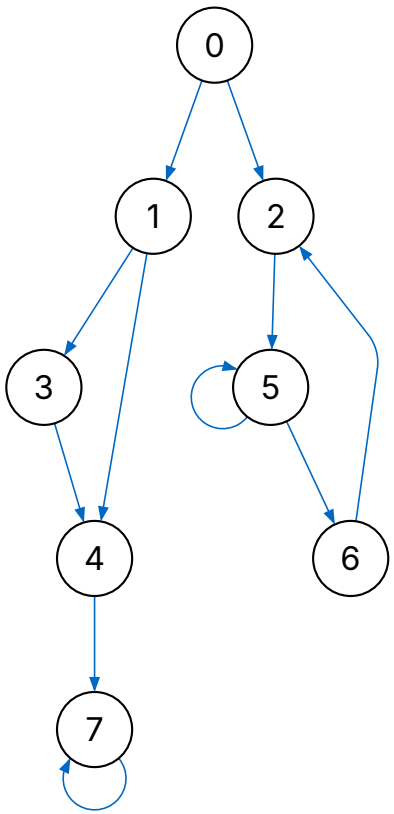
Dessiner le graphe de cette matrice d'adjacence.

```
adjacence = [
    [0, 1, 1, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 0, 0, 0],
    [0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 1, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 1, 1, 0],
    [0, 0, 1, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1],
]
```

Exercice : dessiner le graphe de cette matrice d'adjacence

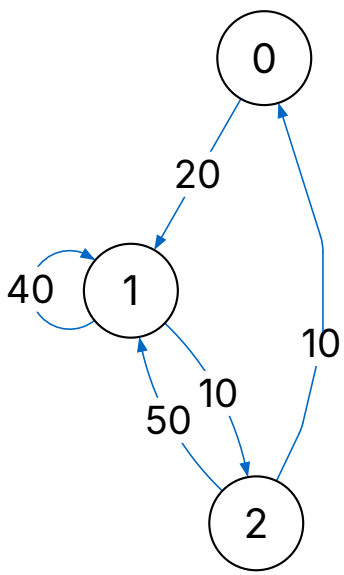

```
adjacence = [
    [0, 1, 1, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 0, 0, 0],
    [0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 1, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 1, 1, 0],
    [0, 0, 1, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1],
]
```

	0	1	2	3	4	5	6	7
0	0	**1**	**1**	0	0	0	0	0
1	0	0	0	**1**	**1**	0	0	0
2	0	0	0	0	0	**1**	0	0
3	0	0	0	0	**1**	0	0	0
4	0	0	0	0	0	0	0	**1**
5	0	0	0	0	0	**1**	**1**	0
6	0	0	**1**	0	0	0	0	0
7	0	0	0	0	0	0	0	**1**



[Graphviz en ligne]

Graphes pondérés : matrice d'adjacence



Graphviz en ligne

- *Le graphe a 3 nœuds, donc une matrice 3 × 3.*
- *Le nombre dans la matrice en position (r, c) indique le poids de l'arc de r à c.*

	0	1	2
0	0	**20**	0
1	0	**40**	**10**
2	**10**	**50**	0

Code Python correspondant :

```
matrice_adjacence = [
    [ 0, 20,  0],  # Nœud 0 -> arc vers le nœud 1
    [ 0, 40, 10],  # Nœud 1 -> arcs vers les nœuds 1 et 2
    [10, 50,  0],  # Nœud 2 -> arcs vers les nœuds 0 et 1
]
```

Sommaire

1 **Tableaux**
Représenter des données matricielles

2 **Graphes**
Différentes représentations informatiques de graphes en utilisant des tableaux

3 **Ensembles**

Ensembles

Un ensemble désigne intuitivement un rassemblement d'objets *distincts* (les éléments de l'ensemble).
Un élément ne peut pas apparaître plus d'une fois dans un ensemble.
Les éléments dans un ensemble ne sont pas ordonnés.

En Python :

```python
amis = {"riri", "fifi"}  # Créer un ensemble à partir d'éléments

s = set()                # Créer un ensemble vide
s.add("a")               # Ajouter des éléments
s.add("b")
s.add("a")
s.add("c")
print(s)
```

```
{'a', 'c', 'b'}
```

Comme prévu : pas de doublons, et pas dans l'ordre d'ajout.

Ensembles : similitudes avec les listes

Beaucoup d'opérations sur les listes s'appliquent aux ensembles.

```
>>> amis = {"riri", "fifi", "loulou"}

>>> len(amis)
3

>>> amis.remove("loulou")
>>> len(amis)
2

>>> amis.remove("loulou")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'loulou'

>>> "loulou" in amis
False

>>> "riri" in amis
True
```

```
>>> if not amis:
...     print("Sniff")
...

>>> for ami in amis:
...     print(ami)
...
fifi
riri

>>> s = {2, 4, 6, 8}
>>> while s:
...     print(s.pop())
...
8
2
4
6
```

Ensembles : contrairement aux listes

À l'inverse, toutes les opérations des listes qui prennent des indices ou qui sous-tendent un ordre ne sont pas possibles avec des ensembles.

```
>>> amis = {"riri", "fifi", "loulou"}

>>> amis[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'set' object is not subscriptable

>>> amis.pop(1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: set.pop() takes no arguments (1 given)

>>> amis.append("moi")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'set' object has no attribute 'append'
```

Ensembles : opérations spécifiques

Créer un ensemble à partir d'une liste :

```
>>> l = ["lait", "oeufs", "pain", "lait"]
>>> set(l)
{'lait', 'oeufs', 'pain'}  # sans doublons
```

Créer une liste à partir d'un ensemble :

```
>>> s = {"lait", "oeufs", "pain"}
>>> l = list(s)
>>> l
['lait', 'oeufs', 'pain']
```

Maintenant qu'on a une liste, on peut avoir des doublons :

```
>>> l.append("lait")
>>> l
['lait', 'oeufs', 'pain', 'lait']
```

Les opérations de théorie des ensembles : union, intersection, différence.

```
>>> a_acheter = {"lait", "oeufs", "pain"}
>>> acheté = {"chips", "chocolat",
...           "pain", "bonbons"}
```

Ce qu'on devait acheter et qu'on a effectivement acheté (ce qui est dans un ensemble **et** dans l'autre) :

```
>>> a_acheter & acheté
{'pain'}
```

Ce qu'on a oublié d'acheter :

```
>>> a_acheter - acheté
{'lait', 'oeufs'}
```

Ce qu'on a acheté par faiblesse morale :

```
>>> acheté - a_acheter
{'chips', 'bonbons', 'chocolat'}
```

Exercice : compter les lettres différentes

Exercice · 8 min

Écrire un programme qui compte le nombre de lettres *différentes* d'un texte.

*Rappel : on peut considérer une chaîne de caractères (le texte) comme une **liste** de caractères.*

```
>>> phrase = "hi marc"
>>> len(phrase)
7

>>> phrase[-1]
'c'

>>> for lettre in phrase:
...     print(lettre)
...
h
i

m
a
r
c
```

référence

Exercice : compter les lettres différentes

*Écrire un programme qui compte le nombre de lettres **différentes** d'un texte.*

Solution sans utiliser les ensembles

- On conserve les lettres déjà vues jusqu'à maintenant dans une liste (initialement vide).
- On sait itérer sur une chaîne de caractères.
- On sait déterminer si un élément est dans une liste en utilisant `élément in liste`.
- On sait compter les éléments d'une liste.

```
>>> texte = "Le vif renard brun saute par-dessus le chien paresseux"   # 54 caractères
>>> lettres_uniques = []
>>> for lettre in texte:
...     if lettre not in lettres_uniques:
...         lettres_uniques.append(lettre)
...
>>> len(lettres_uniques)
20

Juste pour bien comprendre :
>>> lettres_uniques
['L', 'e', ' ', 'v', 'i', 'f', 'r', 'n', 'a', 'd', 'b', 'u', 's', 't', 'p', '-', 'l', 'c', 'h', 'x']
```

Exercice : compter les lettres différentes

*Écrire un programme qui compte le nombre de lettres **différentes** d'un texte.*

Solution en utilisant les ensembles

- On sait qu'un ensemble ne contient pas de doublons.
- On sait qu'on peut considérer une chaîne de caractères comme une liste de caractères.
- On sait créer un ensemble à partir d'une liste.

```
>>> texte = "Le vif renard brun saute par-dessus le chien paresseux"   # 54 caractères
>>> len(set(texte))
20

Juste pour bien comprendre :
>>> set(texte)
{'L', 'p', 'u', 'd', 'l', 'a', 'b', '-', 'h', 'i', 's', 'e', 'x', 'v', ' ', 'n', 'f', 'r', 't', 'c'}
```

Sources

Contenu

Real Python — Sets in Python
realpython.com

Image et vidéo

Luis Ramírez de Lucena — un diagramme d'échecs, *Repetición de amores y arte de ajedrez*, Salamanque (vers 1497, domaine public)
commons.wikimedia.org

Karl Maksudov — « Oh hi Mark », un extrait de *The Room* (Tommy Wiseau, 2003)
youtube.com